
boolean



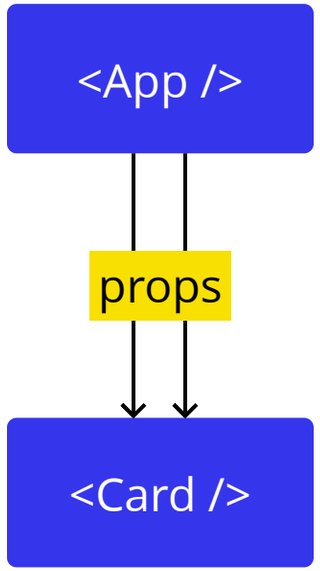
# Props

## Riusabilità alla massima potenza

# Props

## Cosa sono?

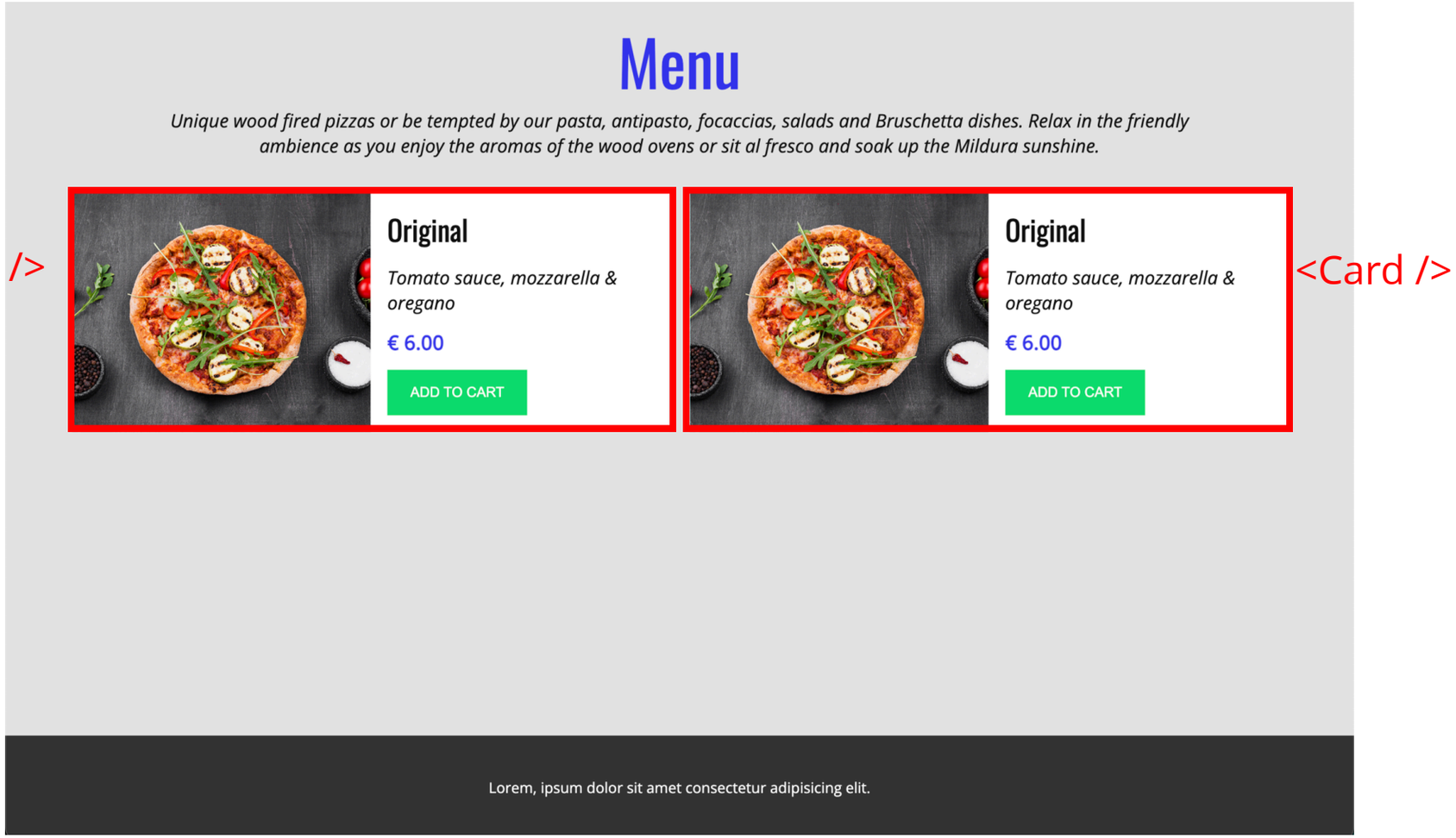
Le **props** sono un meccanismo per passare dati da un componente genitore a un componente figlio, consentendo così il **riutilizzo dei componenti** all'interno dell'applicazione.




React si basa sul **one-way data flow**, il che significa che è possibile passare dati solo da un componente genitore a un componente figlio e non viceversa.

# Component Props

## I componenti ci aiutano a riutilizzare facilmente alcuni elementi...



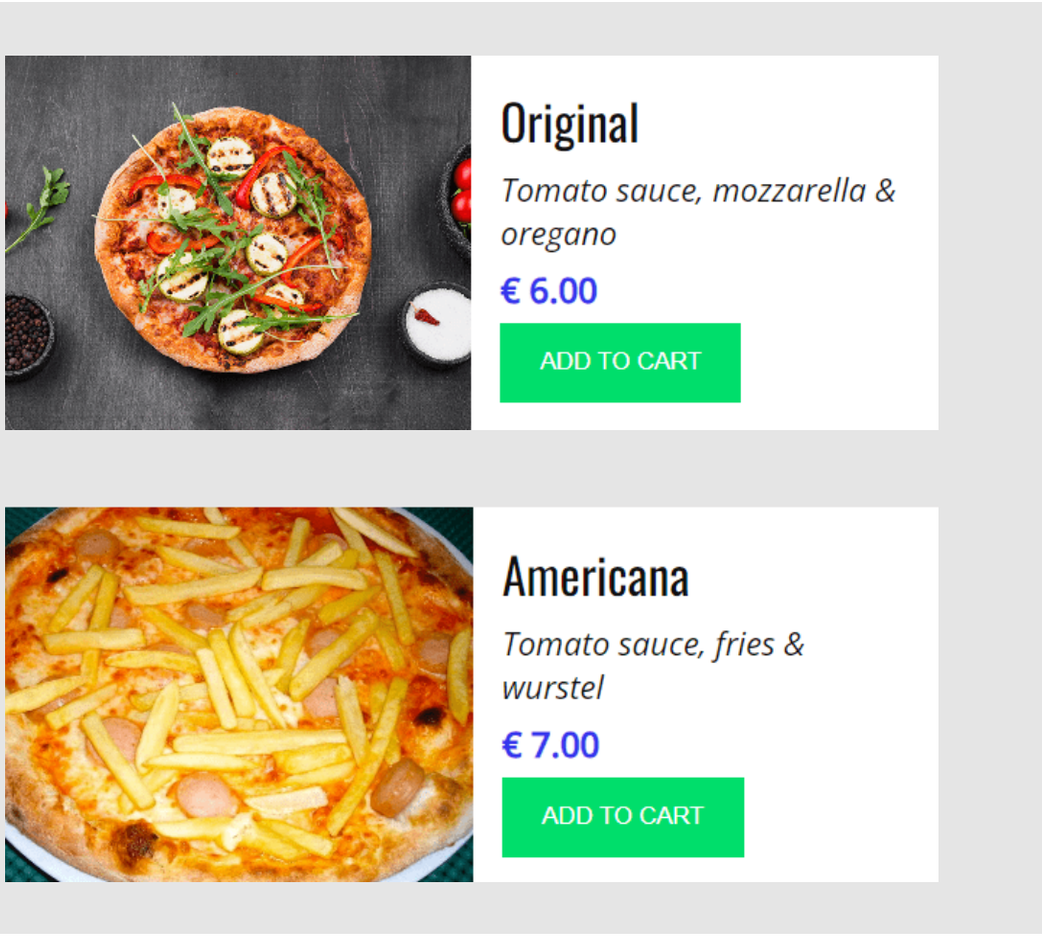
# Component Props

## Ma a che servono nel concreto?

🤔

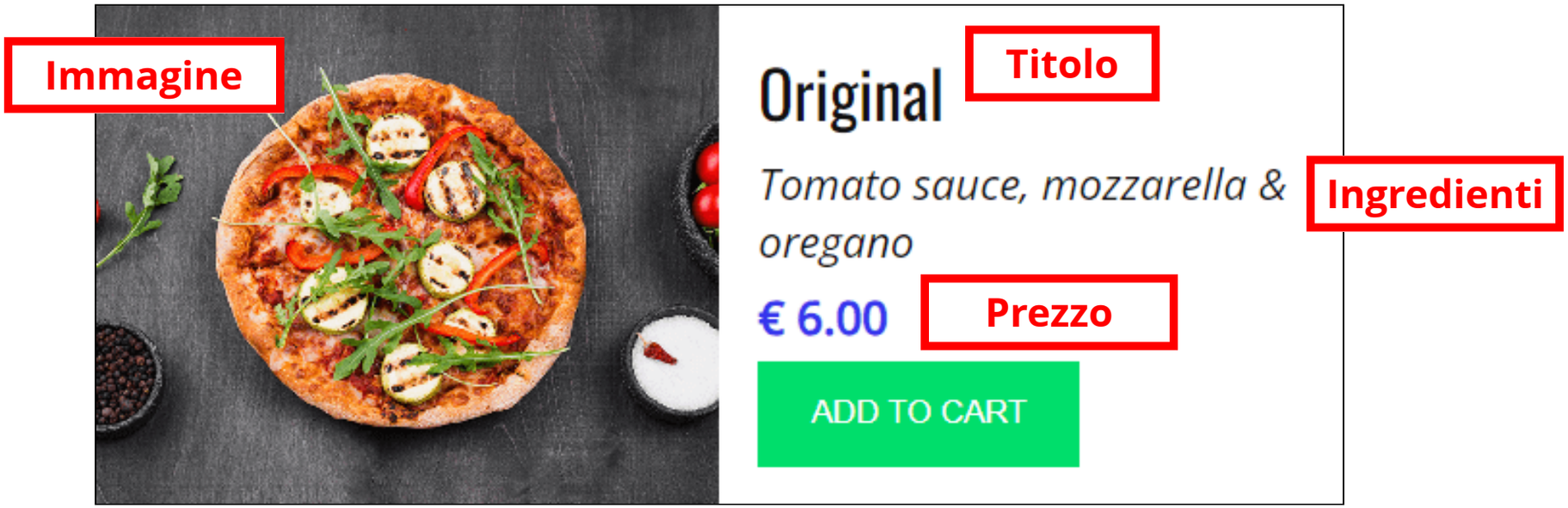
> *E se volessi riutilizzare la stessa card, ma per pizze diverse?*

Servirebbe un sistema per usare lo stesso componente ma con informazioni differenti!



# Component Props

## Scomponiamo la nostra card



# Component Props

## Scomponiamo la nostra card

Il nostro componente è quindi una sorta di modello con dei *"segnaposto"*, detti **props.**

Cambiando i valori delle nostre props, avremo risultati differenti, pur mantenendo l'aspetto ed il comportamento generale del nostro componente!



# Component Props

**Componente**: `Card.jsx`
**Props:**

**`title`**: *"Original"* ,

**`price`**: "€ 6.00",

**`image`**: *"pizza.png"*

**`description`**: *"Tomato sauce, mozzarella & oregano"*

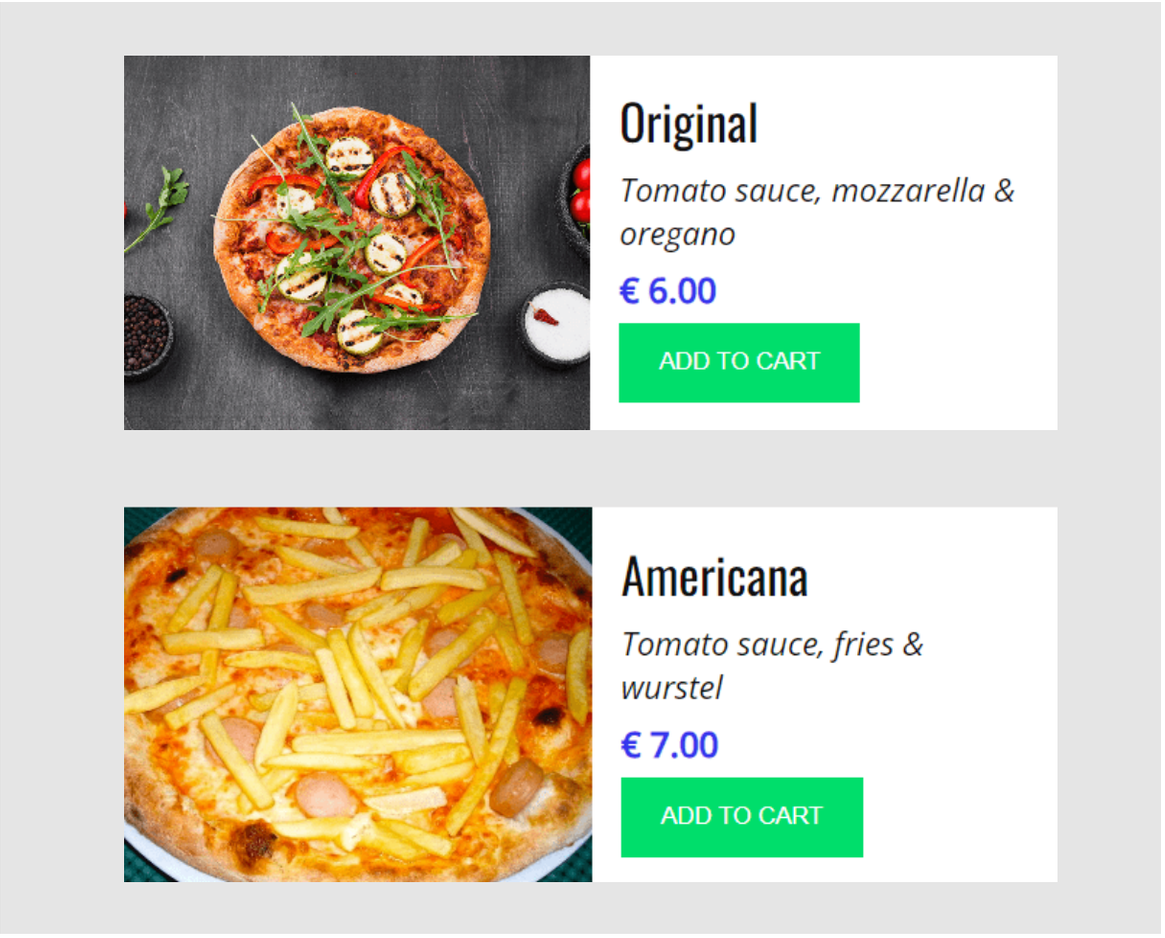
**Componente**: `Card.jsx`
**Props:**

**`title`**: *"Americana"* ,

**`price`**: "€ 7.00",

**`image`**: *"americana.png"*

**`description`**: *"Tomato sauce, fries & wurstel"*



# Props

## In azione

Le **props** vengono trasmesse dal componente genitore al componente figlio attraverso **attributi** HTML. Nel componente figlio, queste props sono disponibili come un **oggetto** che viene passato come parametro alla funzione che definisce il componente.



*In questi esempi stiamo usando soltanto dei valori di tipo stringa, ma possiamo passare ogni tipo di valore e presto lo vedremo!*

```jsx
// App.jsx
import Card from "./components/Card";

export default function App() {
  return (
    <div>
        <Card
            title="Original"
            content="€ 6.00"
            image="pizza.png"
            description="Tomato sauce, mozzarella & origano"/>
        <Card
            title="American"
            content="€ 7.00"
            image="american.png"
            description="Tomato sauce, fries & wurstel" />
    </div>
  )

```

# Props

## In azione

All'interno del componente figlio, le props vengono ricevute sotto forma di parametro all'interno della funzione.

il parametro **`props`** è sempre un oggetto, le cui chiavi rappresentano le singole props.

Notate l'uso delle **parentesi graffe.**

```jsx
// Card.jsx
export default function Card(props) {
  return (
    <div className="card">
      <img src={props.image} alt="picture" />
      <h3>{props.title}</h3>
      <p>{props.description}</p>
      <div className="price">{props.price}</div>
      <button>ADD TO CART</button>
    </div>
  );
}
```



*In questo esempio stiamo semplicemente inserendo i valori ricevuti al posto giusto, ma quando saremo padroni dell'argomento, potremo anche effettuare delle operazioni sui vari dati e trasfrormarli secondo le nostre esigenze!*



**Non capisco ancora il vantaggio**

Tranquilli, con un paio di giorni di tempo e tanti esempi presto capirete...



# LIVE CODING



# Conditional Rendering in JSX

# Conditional Rendering in JSX

## Short-Circuiting e Operatore Ternario

Poiché all'interno del markup **JSX** non è possibile utilizzare istruzioni condizionali come **if...else** o **switch**, possiamo ricorrere a due approcci:

- **Short-Circuiting con operatori logici:**
  - Usando l'operatore **AND** (&&): Restituisce il secondo operando se il primo è vero.
  - Usando l'operatore **OR** (||): Restituisce il primo operando se il primo è vero, altrimenti restituisce il secondo
- **Operatore ternario**
  - Data una condizione, restituisce il primo valore se è vera, altrimenti il secondo valore.

```jsx
const isOpen = true;

export default function Card(props) {
  return (
    <div>
      {isOpen && <Component />}

      {props.content || "Testo alternativo"}

      {isOpen ? <Component1 /> : <Component2 />}
    </div>
  );
}
```

# Classi CSS condizionali

Possiamo utilizzare questi operatori anche per aggiungere o rimuovere classi CSS da un elemento.

Ad esempio, se desideriamo aggiungere la classe **active** a un elemento di un componente in base al valore booleano di una prop, possiamo farlo utilizzando il 'template literal' con l'operatore ternario.

```
export default function MenuItem(props) {
  return (
    <div className={`menu-item ${props.isActive ? 'active' : ''}`}>
      <a href="#">...</a>
    </div>
  );
}
```



# Props

## Children

# Props

## Ripassiamo...

Come abbiamo visto, possiamo usare le props per passare dei dati ai componenti e personalizzarli a seconda dell'occasione.

Per esempio, un componente Alert potrebbe ricevere le props **`type`** e **`message`** per stabilire rispettivamente la classe CSS (e dunque il colore di sfondo) ed un messaggio di testo da mostrare

```jsx
const Alert = (props) => {
  const { type, message } = props;
  return (
    <div className="alert-container">
      <div className={`alert ${type || 'info'}`}>
        <p>{message}</p>
      </div>
    </div>
  );
};

export default Alert;
```

Si è verificato un errore

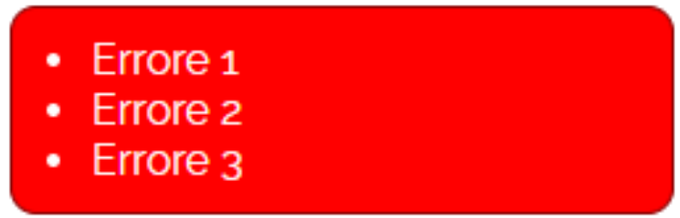
```jsx
<Alert type="error" message="Si è verificato un errore" />
```

# Props

## Un asso nella manica!

Tuttavia, è possibile che per alcuni componenti vorremmo avere più libertà.

Ad esempio, nel nostro componente Alert, in alcuni casi, potremmo preferire mostrare una lista di errori invece di un singolo messaggio!




Ebbene, per ottenere la possibiltà di passare al componente un intero markup come prop, possiamo sfruttare la prop speciale **`children`**

# Props

## Un asso nella manica!

La prop **`children`** è una prop speciale che rappresenta il contenuto tra il tag di apertura e di chiusura del componente!

Pertanto, se quando usiamo il componente, inseriamo tra i suoi tag del markup, questo verrà inserito al posto della prop **`children`** all'interno del componente!

```jsx
const Alert = (props) => {
  const { type, children } = props;
  return (
    <div className="alert-container">
      <div className={`alert ${type || 'info'}`}>
        {children}
      </div>
    </div>
  );
};

export default Alert;
```

```jsx
<Alert type="error">
  <ul>
    <li>Errore 1</li>
    <li>Errore 2</li>
    <li>Errore 3</li>
  </ul>
</Alert>
```

- Errore 1
- Errore 2
- Errore 3

# ESERCIZIO